Relatório Executivo do Projeto Routify

1 Cabeçalho do projeto

Campo de identificação	Preenchimento do grupo
Nome do projeto	Routify: Otimizador de Rotas de Entrega
Problema computacional	Calcular rotas eficientes em uma rede logística, reduzindo distância percorrida, tempo de deslocamento e custo operacional.
Conceitos matemáticos	Teoria de grafos, plano cartesiano, matriz de adjacência, distância euclidiana, caminhos mínimos, funções de custo e heurística.
Integrantes e RA	Vitória RA: 8261112860 | Eduardo RA: 826176413 | João RA: 826224097

2. Resumo executivo

O Routify é uma solução computacional para representar pontos de entrega como vértices de um grafo ponderado e calcular rotas entre origem e destino. O sistema atende entregadores, transportadoras e operações de distribuição que ainda definem percursos manualmente. Seu diferencial é converter coordenadas e conexões em uma matriz de adjacência, comparar estratégias de busca e indicar a rota com menor custo conforme o critério selecionado. A primeira parcial consolida o problema, a modelagem, a arquitetura, o backlog e os critérios de validação que orientarão a implementação.

3. Contexto e objetivo

Rotas escolhidas sem apoio analítico podem aumentar quilometragem, consumo de combustível, atrasos e inconsistência operacional. O Routify busca apoiar a tomada de decisão ao estruturar a rede de entregas em um modelo matemático reproduzível. O objetivo principal é calcular e visualizar percursos eficientes, permitindo comparar quantidade de conexões, distância total, custo acumulado e desempenho dos algoritmos sobre o mesmo cenário de teste.

4. Escopo inicial do projeto

Objetivos específicos

a. Modelar pontos de distribuição e entrega como vértices e as ligações como arestas ponderadas.
b. Converter coordenadas do plano cartesiano em distâncias e organizar os dados em matriz de adjacência.
c. Implementar BFS e DFS para explorar a conectividade e Dijkstra e A* para calcular caminhos de menor custo.
d. Construir interface web para cadastrar ou visualizar a rede, selecionar origem e destino e apresentar a rota.
e. Criar cenários de teste, conferir os resultados por cálculo manual e registrar evidências técnicas no GitHub.

Critérios de qualidade e aceitação

Critério	Evidência	Condição de aceite
Exatidão matemática	Comparação com cálculo manual e casos controlados	Dijkstra e A* retornam o menor custo com pesos não negativos
Comportamento algorítmico	Caminho e métricas registrados em cada execução	BFS minimiza conexões; DFS retorna caminho válido sem promessa de ótimo
Confiabilidade	Testes unitários do núcleo e testes funcionais da interface	Cenários definidos executam sem erro e produzem resultados reproduzíveis
Manutenibilidade	Código modular, versionado e documentado	Modelo, algoritmos, interface e testes permanecem separados

5. Tecnologias e arquitetura computacional

O núcleo matemático será desenvolvido em Python 3.12, com NetworkX para representação e conferência dos grafos, NumPy para matrizes e cálculos numéricos, Pandas para organização dos resultados, Matplotlib para visualizações e pytest para testes. A experiência de uso será construída em HTML, CSS e JavaScript, conforme o protótipo apresentado pelo squad. A arquitetura será modular, separando modelo do grafo, algoritmos, serviço de cálculo, interface, testes e documentação. O VS Code apoiará o desenvolvimento, o Figma a prototipação e o GitHub o versionamento e a revisão colaborativa.

6. Organização do grupo e papéis técnicos

Integrante	Papel	Responsabilidade principal
Vitória	Líder da squad	Coordenar escopo, cronograma, backlog, integrações e remoção de impedimentos.
Eduardo	Analista matemático	Formalizar coordenadas, matrizes, pesos, funções de custo e validação dos resultados.
Henrique	Implementador	Desenvolver estruturas, algoritmos, interface e correções técnicas.
João	Documentador e apresentador	Organizar README, evidências, decisões técnicas e comunicação dos resultados.

7. Planejamento inicial e backlog

O desenvolvimento foi organizado em quatro ciclos: fundação e modelagem matemática; prototipação e estruturação do repositório; implementação e testes; integração, refinamento e entrega. O backlog prioriza cadastro do grafo, coordenadas, matriz de adjacência, seleção de origem e destino, execução de BFS, DFS, Dijkstra e A*, comparação de métricas, visualização da rota e testes. Na primeira sprint, o grupo concluirá o modelo de dados, definirá casos de referência e criará a base compartilhada pelos algoritmos.

Ciclo	Período	Foco	Entregável principal
1	Set 2026	Fundação	Requisitos, grafo, plano cartesiano, matrizes e repositório
2	Out 2026	Prototipação	Telas em HTML e CSS, README e início da codificação
3	Nov 2026	Implementação	Algoritmos, integração dos módulos e testes de rota
4	Dez 2026	Entrega	Refinamento, relatório técnico e apresentação final

8. Próximos passos e cronograma da segunda parcial

Ação	Prazo	Evidência esperada
Concluir o modelo do grafo, critérios de comparação e casos de teste	04/10/2026	Matriz, diagrama e cálculos de referência
Implementar a estrutura do grafo e as buscas BFS e DFS	18/10/2026	Código versionado e testes iniciais
Implementar Dijkstra e A* com custo e heurística	01/11/2026	Rotas comparadas e métricas registradas
Integrar interface web e executar testes funcionais e matemáticos	15/11/2026	Protótipo funcional e relatório de testes
Consolidar código, documentação e evidências da segunda parcial	30/11/2026	Pacote revisado para submissão

Síntese da modelagem matemática

A rede será representada por G = (V, E), em que V contém os pontos de entrega e E contém as ligações possíveis. Para coordenadas (x1, y1) e (x2, y2), a distância euclidiana é $d = \sqrt{((x2 - x1)^2 + (y2 - y1)^2)}$. Os valores alimentam a matriz de adjacência ponderada. Dijkstra minimiza o custo acumulado; A* acrescenta a heurística h(n), estimativa entre o nó atual e o destino. BFS e DFS serão utilizados como referências de exploração e conectividade, respeitando suas limitações em grafos ponderados.